

Rigorous, Reproducible LLM Inference Benchmarking on a Single-GPU Laptop with Kubernetes

Executive summary

A strong “single-GPU laptop + Kubernetes” LLM inference paper should treat **Kubernetes as a controlled experimental factor** (deployment + isolation + observability), and treat **LLM inference as a two-phase system** (prefill vs decode) that must be benchmarked with metrics that reflect user experience (TTFT) and sustained capacity (tokens/sec). vLLM and TGI expose first-class LLM metrics (TTFT, inter-token latency, prefill/decode timing) via Prometheus endpoints, making them ideal for rigorous measurement on constrained hardware. ¹

For a single-node cluster, typical “cluster scaling” becomes **(a)** request routing/ingress overhead, **(b)** batching & queueing policy, and **(c)** controlled “oversubscription” experiments such as NVIDIA GPU time-slicing (with explicit caveats about isolation). Kubernetes device plugins normally do **not** allow GPU overcommit or sharing, so any multi-pod “GPU sharing” must be treated as a distinct experimental condition enabled via operator/device-plugin time-slicing. ²

Reproducibility hinges on: **pinning container image digests** (not tags), capturing the full host + driver + runtime inventory, fixing prompt/token distributions, using warm-up + stable windows, and reporting uncertainty (e.g., bootstrap confidence intervals for p95/p99/TTFT). ³

The most defensible core contribution is a **benchmark harness + experiment matrix** that compares a small set of representative open(-weight) models (0.5B–13B examples), across **streaming vs non-streaming**, **closed-loop vs open-loop traffic**, and **precision/quantization regimes**, with synchronized telemetry from server metrics + GPU metrics + client-side timing. ⁴

Research questions and hypotheses

This set is intentionally **LLM-only**, **single-GPU**, and **Kubernetes-specific**. Each hypothesis is testable with the tools/metrics later in this report.

RQ1 — What is the throughput/latency “knee” of a single-GPU LLM server under Kubernetes?

H1: As closed-loop concurrency increases, request latency percentiles and queue depth will rise sharply after a concurrency threshold determined primarily by KV-cache pressure and batching policy (visible via server and KV-cache utilization metrics). vLLM exposes TTFT, inter-token latency, and e2e latency histograms plus KV-cache utilization metrics; this enables locating the knee with telemetry rather than guesswork. ⁵

RQ2 — How do serving backends differ when prefill dominates vs decode dominates?

H2: Workloads with long prompts (prefill-heavy) will show larger TTFT sensitivity to batching/queue delay parameters than short prompts, while decode-heavy workloads will show more stable TTFT but strong sensitivity in inter-token latency and output token throughput. Both vLLM and TGI explicitly separate prefill/decode in exported metrics, enabling phase-aware comparisons. ⁶

RQ3 — What is the performance cost of “Kubernetes plumbing” (Service/Ingress/Gateway) on a laptop?

H3: For short outputs (e.g., 16–64 tokens), networking and request handling overheads (Ingress/Gateway/sidecars) will measurably inflate p95/p99 relative to direct ClusterIP/NodePort, while long outputs will amortize this overhead. KServe’s “Standard mode” uses standard Kubernetes resources (Deployment/Service/Ingress/Gateway/HPA), making it suitable to isolate these components as controlled variables. ⁷

RQ4 — How do precision and quantization trade capacity vs speed on constrained VRAM?

H4: Quantization primarily increases feasible model/context (capacity), but speed effects depend on kernel/hardware support; some quantization implementations are hardware-dependent. Benchmarking must therefore report both model feasibility (max context, max concurrency before OOM) and throughput/latency, not just one. vLLM documents supported quantization formats and hardware compatibility. ⁸

RQ5 — Can “autoscaling experiments” be meaningful on a single node?

H5: On a single GPU, replica scaling without oversubscription will not increase available GPU compute; meaningful autoscaling experiments are limited to (a) scale-from-zero behavior (platform feature) and (b) GPU time-slicing oversubscription, which changes interference and tail latency rather than capacity isolation. KServe Standard mode notes scale-from-zero limitations for HTTP in that mode; NVIDIA GPU operator time-slicing provides an explicit oversubscription mechanism. ⁹

RQ6 — How do streaming responses change perceived latency metrics?

H6: Streaming will improve perceived responsiveness (TTFT) but can change measurement methodology: TTFT must be measured at the client from the first streamed token event, and server must support streaming semantics. TGI supports an OpenAI-compatible Messages API and streaming; Triton decoupled/streaming requires gRPC bi-directional streaming (HTTP alone is insufficient for decoupled responses). ¹⁰

Environment inventory and baseline capture

A rigorous paper should include an **artifactual “environment snapshot”** so others can replicate (or at least understand deviations). Kubernetes + GPU setups fail in subtle ways; your paper should treat environment capture as first-class, not appendix material. Kubernetes GPU scheduling also has specific semantics (GPU requests/limits), which should be captured and validated in the baseline checklist. ¹¹

Inventory checklist table with commands and repo placeholders

Fill the placeholders exactly once, and commit the command outputs as text files in the artifact repo. (Suggested artifact locations are in the last column.)

Inventory field (placeholder)	What to capture (why it matters)	Exact command(s) to run
<GPU_MODEL>	GPU model affects kernel support, quantization speed, and VRAM capacity	<code>nvidia-smi --query-gpu=name --format=csv,noh</code>
<GPU_UUID>	Stable GPU identifier for multi-GPU laptops / eGPU setups	<code>nvidia-smi --query-gpu=uuid --format=csv,noh</code>
<GPU_VRAM_TOTAL_MB>	Determines max model size and KV cache headroom	<code>nvidia-smi --query-gpu=memory.total --format=csv,noheader,nounits</code>
<GPU_POWER_LIMITS_W>	Needed for energy/token and throttle analysis	<code>nvidia-smi -q -d POWER</code> ¹²
<GPU_TEMP_LOG>	Thermal throttling indicator (critical on laptops)	<code>nvidia-smi -q -d TEMPERATURE -l 5 -f artifact</code> <code>gpu_temp.log</code> ¹²
<NVIDIA_DRIVER_VERSION>	Driver/library mismatches cause crashes or performance shifts	<code>nvidia-smi --query-gpu=driver_version --form</code> ¹²
<NVML_VERSION>	NVML governs power/telemetry programmatic access	<code>nvidia-smi -q sed -n '/NVML Version/,+1p'</code>

Inventory field (placeholder)	What to capture (why it matters)	Exact command(s) to run
<OS_NAME_VERSION>	Kernel/OS affects container runtime and GPU stack	<code>cat /etc/os-release</code>
<KERNEL_VERSION>	Kernel impacts driver compatibility	<code>uname -a</code>
<CPU_MODEL>	CPU can bottleneck tokenization, HTTP routing, TLS	<code>lscpu</code>
<RAM_TOTAL_GB>	Impacts host caching and CPU offloads	<code>free -h</code>
<DISK_TYPE_FREE>	Local disk affects model download/cache reproducibility	<code>df -h</code> and (optional) <code>lsblk</code>
<CONTAINER_RUNTIME>	Containerd/Docker affects GPU runtime wiring	<code>kubectl get node -o jsonpath='{.items[0].status.nodeInfo.containerRuntimeVersion}'</code>
<NVIDIA_CONTAINER_TOOLKIT_VERSION>	Required for GPU containers on Kubernetes/containerd	<code>nvidia-ctk --version dpkg -l grep nvidia-toolkit</code> ¹³
<CONTAINERD_CONFIG>	Confirms NVIDIA runtime is configured for containerd	<code>sudo nvidia-ctk runtime configure --runtime=containerd</code> (or capture <code>/etc/containerd/config.toml</code>) ¹⁴

Inventory field (placeholder)	What to capture (why it matters)	Exact command(s) to run
<K8S_DISTRIBUTION>	e.g., kind/ minikube/k3s/ microk8s— changes networking/ storage	<code>kubectl cluster-info</code> + distro-specific version cmd
<K8S_VERSION>	Kubernetes features & APIs vary with version	<code>kubectl version --short</code>
<NODE_INFO>	Single-node vs multi-node; OS image; kubelet version	<code>kubectl get nodes -o wide</code>
<GPU_RESOURCE_ADVERTISED>	Verifies device plugin registered <code>nvidia.com/ gpu</code>	<code>kubectl describe node sed -n '/Capacity:/,/ grep -E 'nvidia.com/gpu Capacity Allocatab</code>
<DEVICE_PLUGIN_VERSION>	Device plugin behavior affects scheduling & sharing	<code>helm list -A grep -i device-plugin</code> (if Helm) <code>kubectl get ds -A grep -i nvidia</code> ¹⁵
<DCGM_EXPORTER_VERSION>	GPU telemetry schema + sampling interval	<code>helm list -A grep -i dcgm</code> and <code>kubectl get dcgm</code> ¹⁶
<LLM_SERVER_IMAGE_DIGEST>	Reproducibility: tags move; digests don't	<code>docker inspect --format='{{index .RepoDigests</code> or <code>skopeo inspect</code> (document the exact digest used)
<MODEL_ID_AND_REVISION>	HF repos can change; pin revision/ commit	<code>python -c "from huggingface_hub import model print(model_info('<MODEL_ID>').sha)"</code>

Where to fill the “unknowns” (explicit request): keep a single source of truth file `artifacts/ENVIRONMENT.md` containing `<GPU_MODEL>`, `<OS_NAME_VERSION>`, `<K8S_VERSION>`, `<NVIDIA_DRIVER_VERSION>`, `<CONTAINER_RUNTIME>`, and `<LLM_SERVER_IMAGE_DIGEST>`; every

script reads from this file (or a machine-readable `artifacts/env/metadata.yaml`). The reason is that digests are immutable whereas tags are not. ¹⁸

Baseline Kubernetes GPU sanity checks (minimum to report): - Report that GPU extended resources are integer and normally cannot be overcommitted / shared between containers, so one GPU usually implies one GPU-using pod at a time unless time-slicing is enabled. ² - Report that Kubernetes GPU requests and limits must match when specified. ¹⁹

Serving stack survey for Kubernetes LLM serving

Your paper will be strongest if it benchmarks **a small set of conceptually distinct serving paths:**

- **LLM-native engines:** vLLM, TGI, Triton+TensorRT-LLM
- **Kubernetes model-serving control planes:** KServe, Seldon, Ray Serve/KubeRay, IIm-d
- **Generic inference servers** (baseline/ablation): MLServer

Framework/operator/backend comparison table

Stack	What it is (LLM relevance)	Key pros for laptop single-GPU benchmarking	Key cons / confounds	GPU + K8s notes	MI ins
vLLM ²⁰	High-performance LLM engine + OpenAI-compatible HTTP server; exposes <code>/metrics</code> with TTFT, inter-token latency, prefill/decode, KV cache usage	Strong LLM-native metrics; easy reproducible CLI benchmarking (<code>vllm bench serve</code> , request-rate + burstiness); official GPU Docker image; good for TTFT/throughput studies	Needs careful KV-cache tuning (max model len / max seqs / batched tokens); kernel/driver sensitivity	Prometheus endpoint on the OpenAI server; supports <code>--max-num-batched-tokens</code> , <code>--max-num-seqs</code> , GPU memory utilization; use shared memory (<code>/dev/shm</code>) for multiprocess models	Us pin ca en pa v se

Stack	What it is (LLM relevance)	Key pros for laptop single-GPU benchmarking	Key cons / confounds	GPU + K8s notes	Misc
Text Generation Inference 22	Router + model server architecture with batching; OpenAI-compatible "Messages API" and Prometheus <code>/metrics</code>	Clear separation of router and backend; strong exported prefill/decode metrics; supports streaming via OpenAI-compatible schema; official container images	Many tuning knobs (<code>max-total-tokens</code> , batch token limits) directly affect memory and batching; must document them rigorously	<code>/metrics</code> Prometheus endpoint; router buffers and batches requests; launcher CLI defines memory budget via token limits	Us hu ge in se to - - to me
NVIDIA Triton Inference Server 24 + TensorRT-LLM	General inference server + specialized TensorRT-LLM backend with inflight batching and queue delay controls; Prometheus metrics at port 8002	Strong for "systems paper" angle: batching/queue delay (<code>max_queue_delay_microseconds</code>) is explicit and measurable; metrics endpoint is standard; ecosystem tools (<code>perf_analyzer</code>)	Higher setup complexity (engine build pipeline); streaming uses decoupled models requiring gRPC streaming (measurement differs)	Triton metrics default at <code>:</code> <code>8002/metrics</code> ; TRT-LLM backend supports inflight batching; streaming requires decoupled mode and gRPC bidirectional streaming	Sta TR do ste p (g to ap
KServe 26	Kubernetes-native model serving CRDs; supports "Generative Inference" runtime using vLLM (Hugging Face runtime) with OpenAI-compatible endpoints (prefixed)	Best for studying "K8s control plane overhead" + standardized deployment YAML; Standard mode reduces dependencies; integrates metrics and autoscaling primitives	Adds serving layer confounds (routing, CRDs, gateway/ingress behavior); in Standard mode scale-from-zero for HTTP is not supported (document limits)	Standard mode uses Deployment/Service/Ingress or Gateway API + HPA; generative runtime uses vLLM and exposes OpenAI routes like <code>openai/v1/chat/completions</code> by default	Ins Ga a C I en scr

Stack	What it is (LLM relevance)	Key pros for laptop single-GPU benchmarking	Key cons / confounds	GPU + K8s notes	Misc
Seldon ²⁷ (Core 2)	Kubernetes-native serving/orchestration for ML and LLM systems; integrates multiple runtimes	Useful if your paper includes “serving platform” comparisons beyond KServe; supports modular pipelines	Adds additional control plane + product assumptions; less direct for LLM-engine comparisons unless you standardize runtime	Install via Helm charts; choose runtime (e.g., Triton or others) and keep runtime constant for fair comparisons	Min ins
Ray Serve ²⁸ + KubeRay	Distributed serving framework; Ray Serve LLM aligns with vLLM’s OpenAI server; KubeRay provides CRDs/operator	Great for exploring “serving framework overhead” while keeping vLLM engine args comparable; provides Kubernetes examples with vLLM models	More moving parts (Ray runtime, autoscaling semantics); may be heavy for a laptop unless carefully minimized	Ray Serve LLM supports vLLM “engine_kwargs” compatibility; KubeRay installs CRDs/operator via Helm	Ins via Se Ra ca
MLServer ²⁹	High-performance inference server supporting V2 Inference Protocol; has configurable Prometheus metrics port	Useful as a “generic inference baseline” (showing what you lose without LLM-native scheduling/batching)	Not an LLM-optimized engine by itself; LLM performance depends on a custom runtime or external engine	Exposes metrics on a configurable metrics port; supports V2 protocol commonly adopted across serving frameworks	Us ke ru do me se
llm-d ³⁰	Kubernetes-native distributed inference framework integrating gateway + scheduling + vLLM ecosystem; provides Helm-based “well-lit path” guides	Interesting for a 2026-era paper: routing/scheduling/KV offload concepts; explicit guidance for metrics collection; includes scale-to-zero (feature claim)	Designed for production and often multi-node; on a laptop you must scope to single-node features and document what’s disabled	Integrates Gateway API; emphasizes separation of platform vs workload; includes Prometheus PodMonitor patterns	For de inf gu as

Notes on “maturity” in a paper: instead of subjective labels, cite concrete indicators: TGI’s repository states it is used in production at Hugging Face; KServe is documented as joining CNCF and provides stable installation docs; Triton and TensorRT-LLM docs are maintained by NVIDIA; llm-d publishes versioned releases and “tested and benchmarked recipes.” ³¹

Suggested local architecture diagram

```
flowchart LR
    A[Laptop host\nOS + NVIDIA driver + power/thermals] --> B[Local Kubernetes  
(single node)\ncontainerd/Docker + CNI]
    B --> C[NVIDIA Container Toolkit runtime\n(containerd configured)]
    B --> D[NVIDIA device plugin advertises nvidia.com/gpu]
    B --> E[LLM serving Deployment/Pod\n(vLLM or TGI or Triton+TRT-LLM)]
    E --> F[Service / Gateway / Ingress\n(controlled variable)]
    G[Load generator\n(vLLM bench / k6 / vegeta / locust)] --> F
    E --> H[/metrics endpoints\nserver metrics]
    I[DCGM Exporter DaemonSet\nGPU metrics :9400/metrics] --> J[Prometheus]
    H --> J
    J --> K[Grafana / CSV export\nresults artifacts]
    A --> I
```

This architecture is aligned with how vLLM, TGI, and Triton expose Prometheus-compatible metrics endpoints and how DCGM-Exporter exposes GPU telemetry via `/metrics`. ³²

Workloads and load profiles

A single-GPU laptop cannot “paper over” poor workload choices: your model sizes and token distributions must be explicitly tuned to fit VRAM and to stress realistic bottlenecks (prefill, decode, queueing, networking).

Representative open(-weight) models, sizes, and why they’re useful

A pragmatic set that covers **0.5B–13B** scale with explicit published parameter sizes and context properties:

- **0.5B:** `Qwen/Qwen2.5-0.5B-Instruct` (0.49B params; context length published) ³³
- **3B:** `Qwen/Qwen2.5-3B-Instruct` (3.09B params; context length published) ³⁴
- **7B:** `mistralai/Mistral-7B-Instruct-v0.3` (Apache-2.0 licensed; common 7B baseline) ³⁵
- **13B:** `meta-llama/Llama-2-13b-hf` (13B scale; 4k context indicated in model details; Meta license —treat as open-weight, not OSI “open source”) ³⁶

For strict “open source license only” studies, explicitly separate the analysis into an **Apache-2.0 subset** (e.g., Qwen 0.5B, Mistral 7B) and an **open-weight subset** (e.g., Llama 2) and report licensing constraints as part of reproducibility. ³⁷

Workload parameter sweeps (exact parameters to define and sweep)

Define each request as `(prompt_tokens, generate_tokens, streaming_flag, sampling_policy)` and keep the sampling policy deterministic-leaning to reduce output-length variability.

Key workload controls (chosen because the serving stacks explicitly expose or depend on them):

- **Prompt length** (`prompt_tokens`): 32, 128, 512, 2048 (include at least one prefill-heavy point)
- **Generation length** (`generate_tokens`): 32, 128, 512 (include at least one decode-heavy point)
- **Context cap** (`max_model_len / max_total_tokens`): set per model and report; TGI's `--max-total-tokens` is the “memory budget” knob and must be documented. ³⁸
- **Batching controls:**
 - vLLM: `--max-num-batched-tokens`, `--max-num-seqs` (continuous batching capacity knobs) ³⁹
 - TGI: `--max-batch-total-tokens`, `--max-batch-prefill-tokens` (batch token budgets) ⁴⁰
 - Triton+TRT-LLM: `max_queue_delay_microseconds` (trade throughput vs latency) ⁴¹
- **Streaming vs non-streaming**: on for interactive / TTFT studies; off for pure throughput studies. ¹⁰
- **Precision/quantization:**
 - FP16/BF16 (baseline)
 - 8-bit / 4-bit quantization *where supported*; report quant kernel compatibility. ⁸

Determinism note you should explicitly report: even with fixed seeds, GPU operations may be nondeterministic; PyTorch documents deterministic algorithm controls, and NVIDIA notes that atomics/unordered execution can affect determinism in parallel reductions. Your paper should frame deterministic settings as *best effort* and emphasize measuring distributional metrics (percentiles with CI) rather than relying on identical token outputs. ⁴²

Dataset choices: real vs synthetic prompts

Use **both** (and report both), because each answers different questions:

Synthetic prompts (control experiments).

Use controlled token lengths to isolate engine behavior. vLLM benchmarking supports dataset modes such as ShareGPT and HuggingFace datasets, and also supports controlling input/output lengths through benchmark parameters (e.g., `--input-len`, `--output-len` via `vllm bench serve`). ⁴³

Real prompt datasets (ecological validity).

Use a stable public dataset and pin its revision. The vLLM benchmark CLI demonstrates using Hugging Face datasets and ShareGPT-style corpora (e.g., `Aeala/ShareGPT_Vicuna_unfiltered`) for benchmarking. ⁴⁴

Practical guidance: subsample to fixed `N` prompts (e.g., 1k–10k), and report the empirical distribution of prompt lengths.

Load profiles and traffic patterns to test

To produce a convincing benchmarking paper, include **both closed-loop and open-loop** experiments, and explain why. Grafana k6 explicitly distinguishes open-model executors such as constant-arrival-rate. ⁴⁵

Closed-loop (concurrency-controlled) profiles (queueing/knee finding): - Concurrency sweep:

`C ∈ {1, 2, 4, 8, 16, 32}` (upper bound depends on OOM and “knee”)

- Keep request mix fixed; measure TTFT p50/p95/p99 and inter-token latency (ITL) percentiles.

Open-loop (arrival-rate-controlled) profiles (SLO realism): - Constant arrival rate: `λ ∈ {1, 2, 5, 10, 20} req/s` with timeouts, and record drop/error rates. ⁴⁵

- Bursty arrivals: use a burstiness parameter where supported; vLLM’s benchmark CLI exposes `--burstiness` (Gamma distribution) and documents its semantics. ⁴⁶

Streaming vs non-streaming: - For streaming, define TTFT at the client as “time from request send to first token event.” TGI’s OpenAI-compatible Messages API example shows `stream: true`. ⁴⁷

- For Triton streaming, explicitly state whether you used a decoupled model and gRPC streaming; Triton’s decoupled model docs state HTTP cannot be used for decoupled responses and that bi-directional streaming gRPC is required. ⁴⁸

Autoscaling and GPU sharing on a single node (what is feasible and how to do it rigorously)

On a single node with one physical GPU, replica autoscaling is constrained by Kubernetes GPU semantics: extended resources are integer, not overcommittable, and devices cannot be shared between containers by default. ⁴⁹

Feasible autoscaling experiments that still produce publishable insight: - **Scale up/down of HTTP routing path** (replicas for non-GPU sidecars, gateways, routers) while LLM pod count stays fixed at 1. - **HPA based on request metrics (not GPU)** to demonstrate control plane behavior, even if capacity does not increase. - **Scale-from-zero** only if your chosen serving mode supports it; KServe’s Standard mode notes scale-from-zero for HTTP is not supported there (make this a design choice you justify). ⁷ - **GPU time-slicing oversubscription:** NVIDIA GPU Operator supports oversubscription through time-slicing, enabling multiple pods to interleave on one GPU; explicitly note the lack of memory/fault isolation when interpreting tail latencies. ⁵⁰

Benchmarking tools, metrics, and telemetry

A publishable benchmarking methodology should separate: **(a) load generation & client timing**, **(b) server-exported LLM metrics**, and **(c) GPU/node telemetry**.

Primary metrics to report (LLM-specific)

Use definitions consistent across stacks:

- **TTFT (time to first token)**: server metric if available; client metric for streaming also required. vLLM exports `vllm:time_to_first_token_seconds`; GenAI-Perf lists TTFT as a first-class metric; TGI exports prefill/decode-related durations. ⁵¹
- **Inter-token latency (ITL) / time per output token (TPOT)**: vLLM exports `vllm:inter_token_latency_seconds`; GenAI-Perf reports inter token latency; TGI exports decode duration histograms. ⁵¹
- **Output token throughput (tok/s) and request throughput (req/s)**: vLLM exposes token counters; GenAI-Perf reports token throughput; Triton perf tools report throughput/latency windows. ⁵²
- **Tail latency percentiles**: p95/p99 for TTFT and e2e request latency. Triton `perf_analyzer` supports stabilizing based on a chosen percentile. ⁵³
- **Queue depth / waiting**: vLLM has `num_requests_waiting`; TGI has `tgi_queue_size`; these are crucial for interpreting tail latency. ⁶
- **GPU utilization and memory**: DCGM-Exporter metrics + `nvidia-smi` logs; report SM utilization and VRAM usage. ⁵⁴
- **Power/energy proxies**: record power draw over time and report joules/token or watt-seconds/token. `nvidia-smi` provides power query and loop logging; NVML is the underlying library for programmatic queries. ⁵⁵

Tooling: what to use for which claim

vLLM benchmark CLI (for vLLM and OpenAI-compatible servers).

Use it for controlled request-rate and burstiness tests. It documents `--request-rate` and `--burstiness` semantics and supports datasets including Hugging Face datasets and ShareGPT. ⁵⁶

Example (open-loop + burstiness, synthetic lengths, streaming off for throughput stability):

```
# Server (in a K8s pod): vllm serve <MODEL_ID> <ENGINE_ARGS...>

# Client benchmark from your laptop (or a dedicated loadgen pod)
vllm bench serve \
  --backend vllm \
  --model Qwen/Qwen2.5-3B-Instruct \
  --endpoint /v1/chat/completions \
  --dataset-name hf \
  --dataset-path Aeala/ShareGPT_Vicuna_unfiltered \
  --hf-split train \
  --num-prompts 2000 \
  --request-rate 5 \
  --burstiness 1.0 \
  --max-concurrency 16 \
```

```
--save-result \
--result-dir artifacts/results/vllm
```

Triton perf_analyzer (for generic throughput/latency curves).

Use it to produce reproducible latency-throughput curves with stabilized measurement windows and percentile stabilization (`--percentile`). ⁵³

Example (concurrency sweep, stabilize on p95):

```
perf_analyzer -m <model_name_in_triton_repo> \
--concurrency-range 1:16:1 \
--measurement-interval 5000 \
--percentile=95
```

GenAI-Perf (NVIDIA LLM tool; note deprecation).

If you include it, you must cite that it is being phased out and reference its outputs as still useful for TTFT/ITL/token throughput. It also documents the NGC container `nvcr.io/nvidia/tritonserver:<RELEASE>-trtllm-python-py3` and an end-to-end example flow. ⁵⁷

Open-loop load generation (k6).

k6 constant-arrival-rate is explicitly an open-model executor (arrivals independent of response), which helps avoid coordinated omission in latency measurement when the system saturates. ⁴⁵

Example k6 scenario skeleton (OpenAI-compatible route; adapt URL/path):

```
import http from "k6/http";

export const options = {
  scenarios: {
    car: {
      executor: "constant-arrival-rate",
      rate: 5, // iterations per second
      timeUnit: "1s",
      duration: "3m",
      preAllocatedVUs: 20,
      maxVUs: 200
    }
  }
};

export default function () {
  const url = "http://<SERVICE_HOST>/v1/chat/completions";
  const payload = JSON.stringify({
    model: "tgi",
    messages: [{ role: "user", content: "Hello" }],
  });
```

```

    stream: false,
    max_tokens: 128
  });
  http.post(url, payload, { headers: { "Content-Type": "application/json" } });
}

```

Constant-rate, simple CLI load (Vegeta).

Vegeta is designed to drill HTTP services at a constant request rate. ⁵⁸

Use it when you want transparent, scriptable open-loop arrival, but note it is not LLM-aware (you must define payloads and compute token metrics separately).

Scenario scripting (Locust).

Use Locust when you need request-mix logic. Locust documents how user count maps to concurrency, and provides wait-time strategies (e.g., `constant_throughput`) that can approximate target iteration rates.

⁵⁹

Telemetry collection: Prometheus + DCGM + server metrics

Server metrics endpoints to scrape: - vLLM: `/metrics` on the OpenAI server port (example uses `curl http://0.0.0.0:8000/metrics`). ⁶⁰

- TGI: `/metrics` Prometheus endpoint with metrics including queue size and prefill/decode durations. ⁶¹

- Triton: Prometheus metrics at `http://localhost:8002/metrics` by default. ⁶²

- DCGM-Exporter: Helm chart defaults to service port **9400** and exposes metrics at `/metrics`. ⁶³
OpenMetrics endpoints conventionally use `/metrics`. ⁶⁴

Example Prometheus scrape config (plain Prometheus, minimal).

Prometheus `scrape_config` supports static targets and service discovery. ⁶⁵

```

scrape_configs:
- job_name: "vllm"
  metrics_path: /metrics
  static_configs:
    - targets: ["vllm.default.svc.cluster.local:8000"]

- job_name: "tgi"
  metrics_path: /metrics
  static_configs:
    - targets: ["tgi.default.svc.cluster.local:80"]

- job_name: "triton"
  metrics_path: /metrics
  static_configs:
    - targets: ["triton.default.svc.cluster.local:8002"]

- job_name: "dcmg"

```

```
metrics_path: /metrics
static_configs:
  - targets: ["dcgm-exporter.gpu-operator.svc.cluster.local:9400"]
```

If you instead use Prometheus Operator, use `ServiceMonitor` / `PodMonitor` CRDs; the Prometheus Operator docs explain these objects' role in selecting scrape targets. ⁶⁶

Reproducibility, experiment matrix, and deliverables

Required reproducibility practices (what reviewers look for)

Pin everything that can move: - **Container images by digest** (not tags): digests are immutable whereas tags can change; Docker explicitly frames digests as content-addressable identifiers. ¹⁷

- For Kubernetes, document `imagePullPolicy` behavior; `Always` resolves tags to digests and may still reuse cached content with the same digest. ⁶⁷

- **Model revisions:** record Hugging Face commit SHA/revision for each model (and dataset), not just the model ID. - **Server configuration:** vLLM engine args (`max-num-seqs`, `max-num-batched-tokens`, `gpu-memory-utilization`) and TGI launcher args (`max-total-tokens`, batch token limits) materially change behavior and must be recorded. ⁶⁸

- **GPU sharing mode:** explicitly record whether GPU time-slicing was enabled; NVIDIA documents this as oversubscription via time-slicing. ⁶⁹

Warm-up and stable measurement windows: - Triton `perf_analyzer` measures over a time window and repeats until values stabilize; it can stabilize based on a chosen percentile (e.g., p95). This is a strong methodological anchor you can generalize: run consistent warm-up + stable measurement windows across all stacks. ⁵³

Replication strategy and uncertainty: - Run each condition $R \geq 5$ times (practical on a laptop) and report median + bootstrap CI for TTFT p95/p99 and token throughput. SciPy documents `scipy.stats.bootstrap` confidence intervals and coverage interpretation; cite it when describing your statistical method. ⁷⁰

Experiment matrix (grid)

A laptop paper should keep the matrix small but factorial enough to support conclusions. Below is a compact grid that still covers the key dimensions.

Scenario ID	Prompt tokens	Generate tokens	Streaming	Dataset	Load model	Primary metrics	Sweep variables
S1 Interactive-short	128	128	On	Synthetic prompts with fixed token length	Closed-loop	TTFT p50/p95/p99, ITL p95/p99, queue size	Concurrency

Scenario ID	Prompt tokens	Generate tokens	Streaming	Dataset	Load model	Primary metrics	Sweep variables
S2 Prefill-heavy	2048	128	On	Real prompts filtered to long inputs	Closed-loop	TTFT, prefill time, queue growth	Concurrency
S3 Decode-heavy	128	512	On	Synthetic	Closed-loop	ITL, output tok/s, GPU util	Concurrency
S4 Throughput-max	512	128	Off	Real prompt corpus	Open-loop	req/s @ error<1%, tok/s, p99 e2e	Arrival rate λ sweep; burstiness
S5 Kubernetes plumbing	128	64	On	Synthetic	Closed-loop	Added p99 due to ingress/gateway	Routing mode (ClusterIP vs Gateway/ Ingress)
S6 Oversubscription	128	128	On	Synthetic	Closed-loop	Tail latency inflation, interference	Time-slicing replicas

Source-linked justification: vLLM's metrics include TTFT, inter-token latency, and phase metrics; TGI exports prefill/decode durations and queue size; Triton metrics exist at `:8002/metrics`; vLLM benchmark CLI supports request rate and burstiness; NVIDIA GPU Operator documents time-slicing for oversubscription. ⁷¹

Recommended baseline Kubernetes manifests (snippets)

These are intentionally minimal; the paper should publish the full versions in the artifact repo with digests pinned.

Baseline: GPU-enabled Deployment template (vLLM example).

vLLM provides an official Docker image and recommends using IPC host or adequate shared memory; it also shows mounting the Hugging Face cache and using `HF_TOKEN`. ⁷²

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vllm
spec:
  replicas: 1
```



```

selector:
  matchLabels: { app: vllm }
template:
  metadata:
    labels: { app: vllm }
  spec:
    containers:
      - name: vllm
        image: vllm/vllm-openai:<TAG>@sha256:<DIGEST>
        args:
          - "--model"
          - "Qwen/Qwen2.5-3B-Instruct"
          - "--dtype"
          - "auto"
          - "--max-model-len"
          - "32768"
          - "--gpu-memory-utilization"
          - "0.90"
          - "--max-num-seqs"
          - "64"
          - "--max-num-batched-tokens"
          - "8192"
        ports:
          - containerPort: 8000
        env:
          - name: HF_TOKEN
            valueFrom:
              secretKeyRef:
                name: hf-token
                key: token
        resources:
          limits:
            cpu: "8"
            memory: "24Gi"
            nvidia.com/gpu: 1
          requests:
            cpu: "4"
            memory: "16Gi"
            nvidia.com/gpu: 1
        volumeMounts:
          - name: shm
            mountPath: /dev/shm
          - name: hf-cache
            mountPath: /root/.cache/huggingface
    volumes:
      - name: shm
        emptyDir:
          medium: Memory

```

```

        sizeLimit: "4Gi"
- name: hf-cache
  hostPath:
    path: /home/<USER>/cache/huggingface
    type: DirectoryOrCreate

```

GPU scheduling rule you should cite in your paper text: GPU requests and limits must match when both are set. ¹⁹

TGI baseline (showing “memory budget” knobs).

TGI’s launcher args define maximum input tokens and, critically, `--max-total-tokens` as a “memory budget” for requests, and it provides batch token caps. ⁷³

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: tgi
spec:
  replicas: 1
  selector:
    matchLabels: { app: tgi }
  template:
    metadata:
      labels: { app: tgi }
    spec:
      containers:
        - name: tgi
          image: ghcr.io/huggingface/text-generation-
inference:<VERSION>@sha256:<DIGEST>
          args:
            - "--model-id"
            - "mistralai/Mistral-7B-Instruct-v0.3"
            - "--max-input-tokens"
            - "2048"
            - "--max-total-tokens"
            - "4096"
            - "--max-batch-total-tokens"
            - "8192"
            - "--max-batch-prefill-tokens"
            - "2098"
          ports:
            - containerPort: 80
          resources:
            limits:
              nvidia.com/gpu: 1
              memory: "28Gi"

```

```

      cpu: "8"
    requests:
      nvidia.com/gpu: 1
      memory: "20Gi"
      cpu: "4"
    volumeMounts:
      - name: shm
        mountPath: /dev/shm
  volumes:
    - name: shm
      emptyDir:
        medium: Memory
        sizeLimit: "4Gi"

```

Triton+TensorRT-LLM: key config “knobs” you should expose as experimental variables.

The TensorRT-LLM backend documents `max_queue_delay_microseconds` as a way to increase the chance that nearby requests are scheduled together, and documents `decoupled_mode` requirements for streaming semantics; Triton metrics are at `:8002/metrics`. ⁷⁴

Artifact repo deliverables (explicit “where to fill unknowns” + what to publish)

A concise but complete artifact set should include:

- `artifacts/ENVIRONMENT.md` (fill: GPU model, OS, K8s version, driver, runtime, image digests, model SHAs)
- `deploy/` Kubernetes YAMLs with `@sha256:<digest>` image references and resource limits
- `workloads/` JSON for prompt sets + a script that computes token length histograms
- `bench/` scripts for each tool (`vllm bench`, `k6`, `perf_analyzer`) that write CSV/JSON outputs
- `metrics/` Prometheus scrape config or `ServiceMonitor` manifests and DCGM exporter values
- `results/` raw outputs + `analysis/` notebook or scripts that generate the paper’s tables/figures

This aligns with digest immutability guidance and K8s pull policy semantics for reproducibility. ¹⁸

CI automation sketch (GitHub Actions snippet)

Use CI for linting, manifest validation, and lightweight “smoke” runs (CPU-only), while the full GPU benchmark suite runs locally or on a dedicated GPU runner. GitHub documents that workflows live in `.github/workflows` and use YAML syntax. ⁷⁵

```

name: reproducibility-checks
on:
  pull_request:
  push:
    branches: [ main ]

jobs:

```

```

validate-artifacts:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Validate required environment fields present
      run: |
        test -f artifacts/ENVIRONMENT.md
        grep -q "<GPU_MODEL>" artifacts/ENVIRONMENT.md && echo "Fill
placeholders before release" && exit 1 || true
    - name: Validate Kubernetes YAML (schema-ish)
      run: |
        # Example: ensure images are pinned to digests
        ! grep -R "image: .*:latest" -n deploy/ || (echo "Avoid :latest"; exit
1)
        grep -R "@sha256:" -n deploy/ || (echo "Pin images to digests"; exit
1)

```

Suggested paper figures/plots

A focused set that maps directly to your hypotheses:

- **Latency-throughput curves** (x: req/s or concurrency; y: p95 TTFT and output tok/s) per backend
- **Stacked phase time** (prefill vs decode) vs prompt length
- **Tail inflation plot** (p99 TTFT) with and without GPU time-slicing
- **GPU telemetry timelines** (utilization, VRAM, power, temperature) over the same test window
- **Queue depth vs tail latency** scatter to show queueing onset

Mermaid experiment timeline Gantt

```

gantt
  title LLM inference benchmarking paper timeline (single-GPU laptop +
Kubernetes)
  dateFormat YYYY-MM-DD
  axisFormat %b %d

  section Environment & baselines
  Capture full inventory + pin images/datasets :a1, 2026-03-15, 7d
  Validate GPU scheduling + metrics endpoints :a2, after a1, 5d

  section Serving stacks
  Deploy vLLM + telemetry :b1, after a2, 5d
  Deploy TGI + telemetry :b2, after b1, 5d
  Deploy Triton+TRT-LLM (if included) :b3, after b2, 7d

  section Benchmark runs
  Closed-loop sweeps (S1-S3) :c1, after b3, 7d

```

Open-loop sweeps + burstiness (S4)	:c2, after c1, 5d
Kubernetes plumbing + oversubscription (S5-S6)	:c3, after c2, 5d
section Analysis & paper artifacts	
Bootstrap CI + tables/figures	:d1, after c3, 7d
Reproducibility package + camera-ready	:d2, after d1, 5d

Pitfalls and mitigations

These are laptop + LLM specific, and should be treated as “threats to validity” in the paper.

OOM and KV-cache blowups (most common).

Mitigation: treat max sequence length as an explicit experimental factor and tune vLLM/TGI batching caps. vLLM has explicit knobs for sequences and batched tokens per iteration; vLLM also exposes KV-cache utilization metrics to confirm you are near capacity rather than speculating. TGI explicitly frames `--max-total-tokens` as the key “memory budget” knob. ⁷⁶

Thermal throttling and power management (laptops).

Mitigation: log temperature and power continuously with `nvidia-smi -q -d POWER,TEMPERATURE -l ...` and report throttle events as validity threats; ensure consistent fan/power profile across runs. `nvidia-smi` documents loop logging and power query options. ⁷⁷

Driver / CUDA mismatches across containers.

Mitigation: pin driver version in the environment snapshot; prefer official images; for vLLM, document whether CUDA compatibility libraries are used and the `VLLM_ENABLE_CUDA_COMPATIBILITY` setting for older drivers (but note limitations). ⁷⁸

Quantization variability and hardware-dependent kernels.

Mitigation: publish the exact quantization method and note its hardware compatibility; vLLM provides supported formats and a compatibility table for quantization kernels. ⁸

Tokenization/process model nondeterminism.

Mitigation: control tokenization and multiprocessing settings; document PyTorch nondeterminism controls (`torch.use_deterministic_algorithms`) and treat identical output strings as non-required; measure distributions instead. Also, document tokenizer process warnings and explicitly set `TOKENIZERS_PARALLELISM` to avoid fork-related issues when applicable (especially if you benchmark offline tokenization pipelines in multiple processes). ⁷⁹

GPU sharing experiments misinterpreted as “more capacity.”

Mitigation: clearly state that default Kubernetes extended resources cannot be overcommitted and devices cannot be shared; when enabling time-slicing, treat it as oversubscription with no memory/fault isolation, and interpret results as interference studies rather than capacity scaling. ²

Kubernetes serving mode changes the meaning of “autoscaling.”

Mitigation: state whether you use KServe Standard vs Knative; Standard mode uses HPA and has minimal

dependencies but does not support HTTP scale-from-zero, which affects what autoscaling claims you can make from your results. 7

1 5 6 28 32 51 52 71 Metrics - vLLM

<https://docs.vllm.ai/en/stable/design/metrics/>

2 14 49 <https://kubernetes.io/docs/concepts/extend-kubernetes/compute-storage-net/device-plugins/>

<https://kubernetes.io/docs/concepts/extend-kubernetes/compute-storage-net/device-plugins/>

3 17 18 29 <https://docs.docker.com/dhi/core-concepts/digests/>

<https://docs.docker.com/dhi/core-concepts/digests/>

4 <https://kserve.github.io/website/docs/model-serving/generative-inference/overview>

<https://kserve.github.io/website/docs/model-serving/generative-inference/overview>

7 9 26 <https://kserve.github.io/website/docs/admin-guide/kubernetes-deployment>

<https://kserve.github.io/website/docs/admin-guide/kubernetes-deployment>

8 <https://docs.vllm.ai/en/latest/features/quantization/>

<https://docs.vllm.ai/en/latest/features/quantization/>

10 47 https://huggingface.co/docs/text-generation-inference/en/messages_api

https://huggingface.co/docs/text-generation-inference/en/messages_api

11 19 <https://kubernetes.io/docs/tasks/manage-gpus/scheduling-gpus/>

<https://kubernetes.io/docs/tasks/manage-gpus/scheduling-gpus/>

12 24 55 77 <https://docs.nvidia.com/deploy/nvidia-smi/>

<https://docs.nvidia.com/deploy/nvidia-smi/>

13 <https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html>

<https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html>

15 <https://github.com/NVIDIA/k8s-device-plugin>

<https://github.com/NVIDIA/k8s-device-plugin>

16 54 DCGM-Exporter

https://docs.nvidia.com/datacenter/dcgm/latest/gpu-telemetry/dcgm-exporter.html?utm_source=chatgpt.com

20 27 50 69 Time-Slicing GPUs in Kubernetes — gpu-operator 22.9.1 ...

https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/22.9.1/gpu-sharing.html?utm_source=chatgpt.com

21 72 78 <https://docs.vllm.ai/en/stable/deployment/docker/>

<https://docs.vllm.ai/en/stable/deployment/docker/>

22 67 <https://kubernetes.io/docs/concepts/containers/images/>

<https://kubernetes.io/docs/concepts/containers/images/>

23 <https://huggingface.co/docs/text-generation-inference/quicktour>

<https://huggingface.co/docs/text-generation-inference/quicktour>

25 62 https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/metrics.html

https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/metrics.html

30 45 <https://grafana.com/docs/k6/latest/using-k6/scenarios/executors/constant-arrival-rate/>
<https://grafana.com/docs/k6/latest/using-k6/scenarios/executors/constant-arrival-rate/>

31 <https://github.com/huggingface/text-generation-inference>
<https://github.com/huggingface/text-generation-inference>

33 37 <https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct>
<https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct>

34 <https://huggingface.co/Qwen/Qwen2.5-3B-Instruct>
<https://huggingface.co/Qwen/Qwen2.5-3B-Instruct>

35 <https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3>
<https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3>

36 <https://huggingface.co/meta-llama/Llama-2-13b-hf>
<https://huggingface.co/meta-llama/Llama-2-13b-hf>

38 40 73 https://huggingface.co/docs/text-generation-inference/basic_tutorials/launcher
https://huggingface.co/docs/text-generation-inference/basic_tutorials/launcher

39 68 76 https://docs.vllm.ai/en/stable/configuration/engine_args/
https://docs.vllm.ai/en/stable/configuration/engine_args/

41 74 https://github.com/triton-inference-server/tensorrtllm_backend/blob/main/docs/model_config.md
https://github.com/triton-inference-server/tensorrtllm_backend/blob/main/docs/model_config.md

42 79 <https://docs.pytorch.org/docs/stable/notes/randomness.html>
<https://docs.pytorch.org/docs/stable/notes/randomness.html>

43 <https://docs.vllm.ai/en/stable/api/vllm/benchmarks/serve/>
<https://docs.vllm.ai/en/stable/api/vllm/benchmarks/serve/>

44 46 56 <https://docs.vllm.ai/en/latest/benchmarking/cli/>
<https://docs.vllm.ai/en/latest/benchmarking/cli/>

48 https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/decoupled_models.html
https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/decoupled_models.html

53 Performance Analyzer — NVIDIA Triton Inference Server
https://docs.nvidia.com/deeplearning/triton-inference-server/archives/triton-inference-server-2310/user-guide/docs/user_guide/perf_analyzer.html

57 https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/perf_analyzer/genai-perf/README.html
https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/perf_analyzer/genai-perf/README.html

58 <https://github.com/tsenart/vegeta>
<https://github.com/tsenart/vegeta>

59 <https://docs.locust.io/en/stable/writing-a-locustfile.html>
<https://docs.locust.io/en/stable/writing-a-locustfile.html>

60 Production Metrics — vLLM
<https://docs.vllm.ai/en/v0.6.6/serving/metrics.html>

- ⁶¹ <https://huggingface.co/docs/text-generation-inference/reference/metrics>
<https://huggingface.co/docs/text-generation-inference/reference/metrics>
- ⁶³ <https://github.com/NVIDIA/dcgm-exporter/blob/main/deployment/values.yaml>
<https://github.com/NVIDIA/dcgm-exporter/blob/main/deployment/values.yaml>
- ⁶⁴ https://prometheus.io/docs/specs/om/open_metrics_spec/
https://prometheus.io/docs/specs/om/open_metrics_spec/
- ⁶⁵ <https://prometheus.io/docs/prometheus/latest/configuration/configuration/>
<https://prometheus.io/docs/prometheus/latest/configuration/configuration/>
- ⁶⁶ <https://prometheus-operator.dev/docs/developer/getting-started/>
<https://prometheus-operator.dev/docs/developer/getting-started/>
- ⁷⁰ <https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.bootstrap.html>
<https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.bootstrap.html>
- ⁷⁵ <https://docs.github.com/actions/using-workflows/workflow-syntax-for-github-actions>
<https://docs.github.com/actions/using-workflows/workflow-syntax-for-github-actions>